

PLATAFORMA REAL-HOLTMONT

Módulo Pre Work Order

Informe detallado de trabajo: de la integración del Pascal Editor al cierre de la Agencia Paperclip

Repositorio: [Luis-Dokkaebi/HOLTMONT-PYTHON](#)

Periodo cubierto: 11 de agosto de 2026 — 9 de septiembre de 2026

Fecha de emisión: 21 de septiembre de 2026

Alcance: 5 pull requests (#132, #134, #174, #175, #176) y 9 commits de producto

Fuente: historial de [git](#), código en [api/](#) e [index.html](#), y suite de pruebas

Resumen en una línea. El módulo Pre Work Order volvió al frontend tras perderse en la migración a FastAPI, y desde entonces se llevó de “abre y se ve” a “calcula, dibuja, estima y reparte trabajo”: plano 2D a escala real, plano 3D que de verdad se dibuja en el editor Pascal, estimación de la Agencia Paperclip aterrizando en las cuatro tablas del formulario, y cada renglón del programa convertido en una tarea con dueño dentro del Tracker.

Índice

- 1. Resumen ejecutivo
- 2. Punto de partida: qué era la integración del Pascal Editor
- 3. Arquitectura del módulo hoy
- 4. Cronología detallada de los siete hitos
- 5. Estado actual: endpoints, archivos y tablas
- 6. Inventario de pruebas del módulo
- 7. Puertas de calidad: evolución de las cifras
- 8. Deuda conocida y pendientes
- 9. Anexo A — Contrato de mensajes con el editor 3D
- 10. Anexo B — Claves, proveedores y diagnóstico
- 11. Anexo C — Índice de commits

1. Resumen ejecutivo

Entre el 11 de agosto y el 9 de septiembre de 2026 el módulo **Pre Work Order** pasó por siete hitos de trabajo, agrupados en cinco pull requests. El hilo conductor es siempre el mismo defecto de fondo: **piezas que decían funcionar sin funcionar**. Un mosaico que no abría nada y no daba error; un botón de “Plano 2D” que devolvía una ilustración inventada en lugar de un plano; un editor 3D que recibía la escena, contestaba el acuse y pintaba un lienzo negro; una Agencia Paperclip que anunciaba “Análisis completado” sobre tablas vacías; una clave de Google configurada en el panel de Vercel que para el proceso no existía.

En cada caso la corrección fue del mismo tipo: eliminar el fallo mudo, sustituirlo por un aviso concreto, y dejar detrás una prueba que mira lo que ve el usuario y no lo que el código cree haber hecho. El caso más claro es la prueba de extremo a extremo del plano 3D: cuenta la geometría dibujada dentro del iframe, no los nodos que llegaron al almacén de estado, porque el bug del lienzo negro pasaba todas las pruebas anteriores.

Lo que hoy funciona y antes no

Capacidad	Antes (11 ago 2026)	Ahora (9 sep 2026)
Abrir el módulo	El mosaico “Pre Work Order” no abría nada, sin error en consola.	Abre desde el mosaico y como pantalla de entrada de <code>WORKORDER_USER</code> .
Plano 2D	Imagen generada por difusión con medidas inventadas, distinta en cada intento.	SVG a escala calculado de la geometría, con cotas y vanos reales, descargable.
Plano 3D	Lienzo negro con cualquier instrucción.	Escena válida contra el esquema del editor; acuse, aviso de piezas descartadas y error visible.
Agencia Paperclip	“Análisis completado” sobre MATERIALES, HERRAMIENTAS y MANO DE OBRA vacías.	Las cuatro tablas se llenan desde el presupuesto estructurado, con respaldo si el Integrador cae.
Programa de la orden	Una sola fila global al Tracker, con INVOLUCRADOS = quien llenó el	Cada renglón se convierte en tarea del responsable real, con su folio, su SLA y

Capacidad	Antes (11 ago 2026)	Ahora (9 sep 2026)
	formulario.	la cotización adjunta.
Configuración	Fallo de clave indistinguible de fallo de librería o de modelo retirado.	Endpoint <code>/api/paperclip/diagnostico</code> y alias de nombres de clave.

Las cifras del periodo

Indicador	11 ago 2026	9 sep 2026	Variación
Pruebas Python (pytest)	1 301	2 570	+1 269
Pruebas GAS	87	97	+10
Cobertura global (piso 60 %)	65.22 %	78.92 %	+13.70 pts
Hallazgos de <code>ruff</code> completo	51	51	sin cambio

Cifras tomadas de la salida de puertas registrada en cada commit. El crecimiento de la suite corresponde a todo el repositorio en ese periodo, no sólo a este módulo.

2. Punto de partida: qué era la integración del Pascal Editor

El **Pascal Editor** es una herramienta open source de diseño arquitectónico 3D construida en React, Next.js, React Three Fiber y WebGPU. La plataforma REAL-HOLTMONT es lo contrario: un monolito Vue 3 sin paso de compilación, incrustado en un único `index.html`, con backend Python (FastAPI). No se podían fusionar; había que hacerlos conversar.

2.1 La decisión de arquitectura

La política de mismo origen del navegador impide leer el DOM o el estado de un iframe de otro dominio. Intentar `iframe.contentWindow.document` lanza un error de CORS de inmediato. La decisión documentada en `PASCAL_EDITOR_INTEGRATION.md` fue, por tanto:

- **Fork propio del editor** (`holtmont-3d-editor`), desplegado aparte —Next.js necesita su propio build de Node—, para poder modificar su código fuente.
- **Mensajería bidireccional** con `window.postMessage`: el único canal legítimo entre el iframe y la página que lo contiene.
- **Un puente en el fork** (hoy `apps/editor/app/holtmont-bridge.tsx`) que exporta la escena hacia la plataforma y acepta escenas guardadas antes.

2.2 Los tres pasos del parche original

`PARCHE_PASCAL_EDITOR.md` conserva como historia el parche mínimo con el que arrancó la integración: un botón `HoltmontExportButton`, su inserción en la barra de herramientas del visor, y un `useEffect` en el layout que escucha los diseños entrantes y avisa al padre con `PASCAL_READY`. Ese parche está aplicado y superado: el puente de hoy además **valida** cada nodo contra el esquema del editor, **contesta qué pasó** con acuse o error, y **resuelve los muebles** contra el catálogo de modelos 3D que vive del lado del editor.

Por qué la validación importa. `useScene.setScene()` del editor no valida nada: guarda el objeto tal cual. Un nodo con un campo de la forma equivocada se importaba sin quejarse y reventaba después, dentro de un `useFrame`, matando el bucle de render de react-three-fiber. Ese —y no otro— era el Plano 3D en negro.

3. Arquitectura del módulo hoy

```
+----- NAVEGADOR -----+
| index.html  (Vue 3, sin paso de compilacion) |
|   |- Formulario Pre Work Order  (bloques A ... H) |
|   |- Boton  Agencia Paperclip  ---+ |
|   |- Boton  Plano 2D           ---+ |
|   '- Vista  Editor 3D Pascal    | |
|       '- <iframe> <-- postMessage --> holtmont-3d-editor |
|                                     (Next.js, en Vercel) |
+-----+
|                                     |
|                                     | api_service.js
|                                     v
+----- FastAPI (api/) -----+
| POST /api/run_paperclip_agency  -> paperclip_agents.py |
| GET  /api/paperclip/diagnostico  -> paperclip_agents.py |
| POST /api/plano_2d               -> services/plano.py   |
| POST /api/savePPC                -> services/work_order.py |
```

```

+-----+
|
|
v
+----- Supabase / MemoryEngine -----+
| wo_materiales  wo_herramientas  wo_mano_obra  wo_equipos |
| wo_programa    wo_viaticos      wo_transporte wo_ingenieria |
| '---> tareas del Tracker, una por responsable |
+-----+

```

3.1 La Agencia Paperclip

Es un grafo de LangGraph con seis nodos, definido en `api/paperclip_agents.py` (1 448 líneas). Cada nodo produce estructura validada con Pydantic, no sólo prosa:

```

START -> levantamiento -> architect -> calculo -> precios -> evaluador -> integrador -> END
                                     ^           |
                                     |           | presupuesto no aprobado
                                     +-----+ (hasta MAX_PRECIOS_REVISIONS)

```

Nodo	Produce	Papel
levantamiento	LevantamientoData	Entiende el encargo del texto libre del cotizador.
architect	ArchitectExtraction → escena Pascal	Extrae medidas y vanos; arma la escena 3D. Si no puede, lo dice.
calculo	CalculoData	Conceptos, cantidades, unidades, personal y equipo.
precios	PreciosData	Precio unitario por concepto, ya validado.
evaluador	PreciosCritique	Critica el presupuesto y decide si hay que rehacerlo.
integrador	StructuredAgencyData	Arma las cuatro tablas del formulario. Ya no es el único camino.

4. Cronología detallada de los siete hitos

Hito 1 · El módulo vuelve al frontend

11 ago 2026 · PR #132 · 861e5e4

El problema

La migración a FastAPI reescribió `index.html` y dejó atrás cuatro piezas del módulo. El backend nunca fue el problema: `/api/config` siempre publicó el módulo con `type: "work_order_form"`, que es justo lo que el frontend anterior consumía. Lo que se perdió fue el lado del navegador:

- `openModule()` no tenía rama para `work_order_form`: el mosaico no abría nada y la cadena de `else if` se agotaba en silencio, **sin error de consola**.
- `loadConfig()` ya no abría el formulario al entrar con `WORKORDER_USER`. Esa cuenta no tiene departamentos ni tracker, así que aterrizaba en una pantalla vacía.
- Los botones de la Agencia Paperclip y del Plano 2D no estaban en el marcado. El JavaScript que los atiende seguía entero, huérfano.
- El editor Pascal había perdido su vista y su `<iframe>`. El pegamento de `postMessage` sobrevivió, pero buscando un id inexistente (`pascalIframe`) y con una forma de mensaje que el fork no entiende.

Lo que se hizo

La inicialización se extrajo a `abrirPreWorkOrder()` —ahora tiene dos llamadores— y limpia el diseño 3D anterior, para que una orden nueva no nazca con el plano de la anterior ya marcado como anexo. La verificación en navegador destapó tres defectos más:

- El mensaje hacia Pascal viaja como `projectData`, no `payload`, y el diseño se desenvuelve con `JSON.parse / JSON.stringify`: del agente llega como cadena y de Pascal llega envuelto en un Proxy reactivo de Vue que `postMessage` no sabe clonar.
- El receptor lee `data.data`, que es lo que el fork manda. Leer `payload` guardaba `undefined` **y aun así marcaba el diseño como anexo**.
- Un reset dejaba `cotizador` como cadena vacía donde el resto del código lo trata como arreglo: agregar un cotizador tras guardar fallaba con “push is not a function”.

Pruebas

`tests/test_prewrite_order_restaurada.py` fija el marcado y los dos contratos de `postMessage`; `test_rbac_config.py` fija el `type` del módulo, que es lo que el frontend rutea y hasta entonces nadie comprobaba.

`index.html` +276 líneas

2 módulos de prueba

El problema

El programa de una orden ya traía responsable y fechas por renglón, pero al Tracker sólo iba **una fila global** de la orden entera, con `INVOLUCRADOS` = quien llenó el formulario en vez de quien ejecuta. Los renglones se quedaban inertes en `wo_programa`, que nadie lee.

Lo que se hizo

- Cada renglón con descripción y responsable se convierte en una tarea en el tracker de esa persona, anclada al folio de su orden. El destino lo da `resolve_worker_sheet` y el guardado va con `como_copia=True`: el mismo camino que la «papa caliente» ya dejó probado, así que una línea de trabajo nunca cae en la tabla de ventas de nadie.
- **Dos descartes deliberados:** un renglón sin responsable no se le asigna al cotizador por descarte —nadie recibe trabajo que no aceptó—, y un renglón sin descripción no genera nada, porque el formulario precrea filas en blanco en las cuatro secciones del programa.
- La marca de sección es `[PWO: <SECCION>]`, con prefijo a propósito: la marca desnuda competía con las de fase que `es_delegacion_de_fase` lee de los corchetes del CONCEPTO.
- **Clasificación del Tracker.** No la mandaba nadie: `CLASIFICACION` caía al default “Media”, que no está en `SLA_LIMITS`, así que toda tarea nacida de una Pre Work Order entraba **sin semáforo y sin SLA**. Ahora hay selector A/AA/AAA con sus días, y una orden sin clasificación válida se rechaza antes de escribir nada: ni folio consumido ni fila a medias.

Pruebas y puertas

52 unitarias (`tests/test_prowork_order_al_tracker.py`), 7 escenarios Gherkin (`tests/features/pre_work_order.feature`) y sus pasos.

Suite completa: 1301 pytest + 87 GAS — en verde
Cobertura: 100 % del diff · 65.22 % global (piso 60 %)

El problema

El botón “Plano 2D (IA)” **no dibujaba un plano**. Llamaba desde el navegador a un generador de imágenes por difusión ([image.pollinations.ai](#)) con el texto de la descripción y mostraba lo que devolviera. Con “habitación de 8 × 5 m con una puerta” eso da una ilustración plausible con medidas inventadas: sin escala, sin cotas, distinta en cada intento —la semilla era aleatoria— y sin ninguna relación con el modelo que abría el Editor 3D. Dos botones de la misma pantalla, alimentados por el mismo campo, contaban cosas distintas.

Lo que se hizo

- **Módulo nuevo** [api/services/plano.py](#) (570 líneas): lee las medidas de la descripción **sin LLM** —“8 x 5”, “8x5”, “8 por 5 metros”, con decimales y con coma, y las cantidades en cifra o en palabra— y dibuja un SVG a escala, con los vanos en su sitio y las cotas reales.
- **Endpoint POST** [/api/plano_2d](#): devuelve el SVG y la escena de Pascal en la misma respuesta, de modo que abrir el 3D enseñe exactamente lo que se acaba de ver en planta.
- **Tres defectos del dibujo** encontrados mirando el resultado y no sólo las pruebas: el título largo se salía del [viewBox](#); una ventana caía encima de una puerta cuando las dos iban al centro de su muro (ahora comparten registro de tramos ocupados); y el arco de barrido se trazaba siempre hacia arriba, de modo que en el muro superior abría hacia fuera del recinto.
- **Se quita el fallo silencioso del agente 3D.** [architect_node](#) caía a una habitación de 4×4 m sin puertas cuando el LLM fallaba, y el frontend lo anunciaba como “3D actualizado”: pedir 8×5 con puerta y recibir 4×4 sin puerta sin ningún aviso es el peor resultado posible, porque parece que funcionó. Ahora intenta leer las medidas del propio texto y sólo si eso tampoco da nada usa la 4×4, **marcada como supuesto y con su aviso**.
- El botón del Editor 3D deja de depender de que alguien haya pulsado antes la Agencia Paperclip, y el plano se puede descargar como SVG: un plano que no se puede adjuntar a la orden no sirve de mucho.

Pruebas y puertas

150 unitarias ([tests/test_plano_2d.py](#)) con una batería de 12 descripciones que comprueba, caso por caso, que la geometría, el dibujo y el modelo 3D son los pedidos; y 5 recorridos de navegador ([tests/test_plano_2d_ui.py](#)) que escriben la descripción, pulsan el botón y verifican las cotas en pantalla.

Suite completa: 1456 pytest + 87 GAS — en verde
Cobertura: 99 % del módulo nuevo · 95 % del diff · 66.59 % global

El problema

El Plano 3D salía **en negro con cualquier instrucción**. La mitad del fallo estaba en este repositorio: `_build_pascal_scene` escribía nodos con la forma de un esquema que el editor no tiene. Como `setScene()` no valida, la escena importaba sin quejarse y el error salía después, dentro del `useFrame` de `DoorSystem`, que mata el bucle de render.

Lo que estaba mal, campo por campo

Elemento	Defecto
Puertas y ventanas	<code>position</code> iba como fracción del muro (0.5). El editor espera <code>[x, y, z]</code> en metros y hace <code>node.position[0]</code> : sobre un número eso da <code>undefined</code> . Faltaban <code>wallId</code> y <code>side</code> .
Escaleras	<code>type: "staircase"</code> no existe (es <code>stair</code>), con <code>steps / fromLevel / toLevel</code> en vez de <code>stepCount / fromLevelId / toLevelId</code> , y sin tramos <code>stair-segment</code> .
Muebles	<code>type: "object"</code> tampoco existe (es <code>item</code>), rotación en grados sobre Z cuando el eje vertical del editor es Y, y sin <code>asset</code> .
Techos	<code>roofType: "gabled"</code> no está en el vocabulario (<code>gable</code>), y <code>pitch / overhang</code> viven en el <code>roof-segment</code> , no en el grupo.
Muros y niveles	Muros sin <code>height</code> , a merced del valor por defecto del visor; niveles colgando del vacío (<code>parentId: null</code>) en vez del edificio.

`tests/test_architect_pascal.py` pasaba con todo eso **porque comprobaba la forma que Python escribía, no la que el editor lee**. Sus aserciones quedaron actualizadas al contrato de verdad.

Del lado del navegador

- `index.html` manda el diseño al recibir `PASCAL_READY` en vez de tras un retraso fijo: si el motor tardaba de más, el mensaje llegaba antes que el oyente y se perdía sin rastro.
- Se atienden `HOLTMONT_3D_IMPORT_ACK` (avisa si el editor dibujó el diseño incompleto, con las piezas que descartó) y `HOLTMONT_3D_IMPORT_ERROR` (lo dice y marca el diseño en error).
- La otra mitad del fallo —el visor daba WebGPU por disponible con `'gpu' in navigator` y montaba una tubería de post-proceso que en WebGL2 no dibuja nada— se corrigió en `holtmont-3d-editor`, junto con la validación de la escena en el puente.

La prueba que faltaba: ¿se ve?

Ni el contrato de la escena ni el intercambio de mensajes responden esa pregunta: el bug del lienzo negro **pasaba las dos cosas**. `tests/test_plano_3d_e2e.py` recorre la cadena entera con el editor de verdad al otro lado del iframe: escribe la descripción, pulsa «Editor 3D Pascal» y **cuenta la geometría que quedó dibujada** (`window.__holtmontProbe()`), no los nodos que llegaron al almacén. Además comprueba que no hubo ninguna excepción en el navegador. Casos: cuarto de 5×4 con puerta; bodega de 12×8 con dos puertas y cuatro ventanas; oficina de 6×4 con puerta y dos ventanas. Se comprobó contra el **build de producción** del editor, no sólo contra el servidor de desarrollo. Cuando el editor no está servido en `HOLTMONT_EDITOR_URL`, la prueba **se salta con el motivo a la vista** en lugar de dar por bueno lo que no se comprobó.

<code>tests/test_arquitectura_pascal_contrato.py</code>	18 casos
<code>tests/test_editor_3d_puente_ui.py</code>	5 recorridos de navegador
<code>tests/test_plano_3d_e2e.py</code>	4 pasan contra el editor real
<code>scripts/generar_escenas_pascal.py</code>	exporta escenas para validarlas allá

Hito 5 · La estimación vuelve a las tablas y la cotización viaja con la orden

8 sep 2026 · PR #175 · 5d18efc

Dos defectos mudos reportados por el dueño el 2026-09-08.

5.1 La Agencia Paperclip ya no llenaba MATERIALES, HERRAMIENTAS ni MANO DE OBRA

`run_paperclip_agency` devuelve `structured_data` con la forma de `StructuredAgencyData: requiredMaterials, toolsRequired, laborTable` y `specialEquipment`, con campos que se llaman igual que las columnas del formulario. `index.html` leía **otro JSON**: `parsedData.materiales, .herramientas, .mano_de_obra, .equipos`. Ninguna de esas claves existe en la respuesta, así que los cuatro `if` daban `undefined`, no se insertaba una sola fila, y el usuario veía el aviso verde «Análisis completado» sobre las tablas vacías de siempre. **Sin excepción y sin consola en rojo.**

En la variante de actividad había un segundo desajuste encima: las filas se marcaban con `contextId` y el modal de recursos filtra por `activityId`.

Las dos rutas comparten ahora `insertarEstimacionPaperclip`, que lee las claves reales, limpia los números que vengan como “\$2,800” y recalcula el total de cada renglón con la misma función que usa la captura a mano. **El aviso deja de ser fijo**: nombra cuántas filas entraron por tabla y avisa en amarillo cuando no entró ninguna. El defecto duró porque el mensaje era el mismo con cero filas que con veinte.

5.2 La persona asignada recibía la orden sin la cotización

El formulario junta cotización y planos en `archivoUrl`, y la fila de cabecera los guarda en `ARCHIVO` — alias de la columna `carpeta`, la que el Tracker pinta como adjunto—. Pero `tareas_de_programa` construía la fila derivada con nueve campos y ninguno era `ARCHIVO`: quien ejecuta el trabajo recibía la tarea con `carpeta = None` y tenía que ir a pedir la cotización a quien la elaboró. La fila derivada hereda ahora `ARCHIVO` de la cabecera. Se confirmó además que la orden se puede asignar a cualquier persona: quien todavía no tiene tracker lo estrena con esa fila.

Pruebas y puertas

`tests/test_paperclip_estimaciones.py` (13): recorrido en el navegador con la agencia simulada por su respuesta literal —construida con los propios modelos Pydantic del agente para que no pueda quedarse con una forma vieja del JSON—. `tests/test_prework_order_adjunta_cotizacion.py` (13): el adjunto en la fila derivada, con varios responsables, con varios documentos, sin documentos, y de punta a punta contra `MemoryEngine`. **Las 26 se vieron en rojo contra el código anterior.**

```
./run_tests.sh → 2516 passed, 10 skipped | GAS: 97/97
ruff --select E9,F63,F7,F82 → All checks passed
xenon --max-average A --max-absolute F → 0
cobertura --cov-fail-under=60 → 78.01 %
```

El problema

El dueño reportó **por segunda vez** que la Agencia Paperclip no escribía nada en las tablas. El arreglo anterior corrigió el formulario y quedó probado con la respuesta del backend escrita a mano. **Ahí estaba el hueco: nadie probó que el backend produjera esa respuesta.**

Qué estaba mal

1. **Toda la estimación dependía de UNA llamada al modelo.** `integrador_node` releía la prosa de los agentes anteriores y volvía a extraer de ella las cuatro tablas con el esquema más grande del archivo —cinco listas anidadas—, **sin reintento** (a diferencia del resto de los nodos, que usan `_invoke_structured`) y **sin respaldo**. Si esa llamada fallaba, o si el modelo devolvía las cinco listas vacías, el usuario veía el formulario intacto.

Lo absurdo es que el dato **ya estaba estructurado** antes de esa llamada: `calculo_node` produce un `CalculoData` y `precios_node` un `PreciosData` con categoría, unidad, cantidad y precio unitario, ya validados. Se serializaban a markdown y se tiraba la estructura para pedirle a un modelo que la volviera a extraer de su propio texto.

Ahora el estado conserva ambas (`calculo_struct`, `precios_struct`) y el integrador arma las tablas **en código** con `estimacion_desde_estructura`. El LLM queda para el caso en que el presupuesto se haya quedado en prosa; si tampoco puede, entran los conceptos del cálculo con costo 0, que es **un pendiente visible en vez de una tabla vacía**.

2. **El informe de ingeniería se perdía entero.** `index.html` leía `result.estructuracion` y `result.conceptoDesc`; la agencia devuelve `levantamiento`, `calculo` y `precios`. La Descripción del Trabajo nunca recibía una línea del análisis.
3. **Cero filas se anunciaba igual viniera de donde viniera.** `estimacion_origen` viaja ahora hasta el aviso: “no había nada que estimar” y “la agencia no pudo armar la estimación” dejan de verse igual desde la pantalla.
4. **Canales no declarados.** `architect_origen` y `architect_aviso` se escribían sin declararse en `PaperclipState`; LangGraph 1.2 descarta en silencio lo escrito a un canal que el esquema no nombra, así que el aviso “este plano es una habitación de ejemplo” no salía del grafo.

Pruebas

`tests/test_estimacion_agencia.py` (24) y `tests/test_estimacion_prework_e2e.py` (6): el recorrido entero, que es lo que faltaba. **El JSON no se escribe a mano:** sale de correr `run_paperclip_agency` y se le entrega al `index.html` real, incluidos los totales con los que sale la cotización y la corrida en la que el Integrador se cae.

7.1 «Paperclip usa apikeys de Google, no de Groq»

La agencia exigía `GROQ_API_KEY` y cortaba antes de ejecutar un solo agente. En un despliegue con clave de Google y sin clave de Groq el endpoint devolvía 500 sobre las tablas intactas. Gemini estaba ahí y sólo se usaba para los agentes de texto: la salida estructurada, que es la que llena las tablas, seguía atada a Groq.

- **Basta una de las dos claves.** `_llms_disponibles` elige proveedor: Groq + Gemini conserva el reparto con el que se afinaron los prompts (Groq formatea, Gemini escribe); sólo Gemini corre los dos lados; sólo Groq igual. Una clave cuya librería no está instalada cuenta como ausente —antes eso llegaba a construir el LLM y reventaba—.
- **La clave puede viajar en la petición.** En Vercel cada invocación es un proceso nuevo y `os.environ` no sobrevive de una a otra, así que una clave guardada desde la pantalla vive en `localStorage`. `PaperclipRequest` la acepta, `api_service.js` la manda y el formulario la lee con `leerGeminiKeyLocal()`.

7.2 La clave se llamaba `Gemini_Key`

Con la captura del panel de Vercel a la vista, dos hallazgos —y uno corrige lo que este mismo hilo había dado por bueno:

1. **La clave de Google estaba puesta y para el proceso no existía.** Las variables de entorno distinguen mayúsculas: `Gemini_Key` y `GEMINI_API_KEY` son dos variables distintas. El código leía sólo la segunda, así que Gemini nunca se usó y la agencia corría entera con Groq, imprimiendo el aviso “GEMINI_API_KEY no detectada” sobre una clave que sí estaba configurada.
2. **GROQ_API_KEY sí estaba configurada**, así que la agencia arrancaba. El commit anterior atribuyó el fallo a su ausencia: eso era falso con ese dato a la vista, y **quedó anotado en el propio commit**. Ese cambio sigue siendo correcto —una de las dos claves basta—, pero la causa del reporte original es la del Hito 6: el Integrador era el único camino a las tablas.

Qué cambia: `ALIAS_CLAVE_GEMINI` y `ALIAS_CLAVE_GROQ` reconocen la clave con los nombres que se usan en la práctica, con el canónico ganando si están las dos —un alias es un rescate, no una segunda fuente de verdad—; y nace `GET /api/paperclip/diagnostico`, que dice qué ve la agencia del despliegue donde corre. **No llama al modelo y nunca devuelve una clave**, sólo su prefijo. Existe porque los tres fallos de configuración de esta agencia —clave con otro nombre, librería fuera del bundle, modelo retirado— **se ven idénticos desde la pantalla**.

Puertas finales del periodo

```
./run_tests.sh → 2570 passed, 10 skipped | GAS: TOTAL 97, PASAN 97
ruff check --select E9,F63,F7,F82 api backend streamlit_cotizador tests
→ All checks passed!
xenon --max-average A --max-absolute F api backend streamlit_cotizador → 0
pytest --cov-branch --cov-fail-under=60 → 78.92 %
```

5. Estado actual: endpoints, archivos y tablas

5.1 Endpoints del módulo

Endpoint	Implementación	Qué hace
POST /api/run_paperclip_agency	api/paperclip_agents.py	Corre el grafo de seis agentes y devuelve las cuatro tablas, los informes y la escena 3D. Acepta <code>geminiKey</code> en la petición.
GET /api/paperclip/diagnostico	api/paperclip_agents.py	Qué claves y librerías ve la agencia, con qué nombre encontró cada clave y qué modelos usaría. No llama al modelo.
POST /api/plano_2d	api/services/plano.py	SVG a escala con cotas reales y la escena de Pascal, en la misma respuesta.
POST /api/savePPC	api/services/work_order.py	Guarda la orden completa: cabecera, los ocho bloques y el reparto de tareas al Tracker.

5.2 Archivos que componen el módulo

Archivo	Líneas	Papel
api/paperclip_agents.py	1 448	Grafo LangGraph, modelos Pydantic, constructor de la escena Pascal, elección de proveedor y diagnóstico.
api/services/plano.py	570	Lectura de medidas sin LLM y dibujo del SVG a escala.
api/services/work_order.py	—	Folio, persistencia de los ocho bloques, <code>tareas_de_programa</code> , clasificación y auditoría de alta.
index.html	—	Formulario Vue, <code>abrirPreWorkOrder()</code> , <code>insertarEstimacionPaperclip</code> , puente <code>postMessage</code> e <code>iframe</code> del editor.
scripts/generar_escenas_pascal.py	116	Exporta escenas de referencia para validarlas contra el Zod real del editor.
PASCAL_EDITOR_INTEGRATION.md · PARCHE_PASCAL_EDITOR.md	—	Guía de arquitectura de la integración y parche del fork (aplicado y superado).

5.3 Tablas de la orden

Cada bloque del formulario tiene su tabla en el esquema relacional; los nombres de hoja y los encabezados en mayúsculas siguen siendo los del formulario y se traducen a tabla y columnas en `TABLAS_WO`:

wo_materiales	wo_herramientas	wo_mano_obra	wo_equipos	wo_programa	wo_viaticos	wo-transporte
wo-ingenieria						

6. Inventario de pruebas del módulo

Doce módulos de prueba cubren hoy la Pre Work Order de punta a punta. La columna «funciones» cuenta las funciones `test_*` del archivo; el total de casos ejecutados es mayor por la parametrización

(por ejemplo, `test_plano_2d.py` ejecuta unos 150 casos desde 52 funciones).

Módulo	Funciones	Qué garantiza
<code>test_prewrite_order_restaurada.py</code>	12	El marcado del módulo y los dos contratos de <code>postMessage</code> .
<code>test_prewrite_order_al_tracker.py</code>	26	El reparto del programa, los descartes deliberados y la clasificación con SLA.
<code>test_plano_2d.py</code>	52	Geometría, dibujo y modelo 3D para 12 descripciones distintas.
<code>test_plano_2d_ui.py</code>	5	Recorridos de navegador: escribir, pulsar y ver las cotas en pantalla.
<code>test_architect_pascal.py</code>	20	El agente arquitecto y su extracción de medidas.
<code>test_arquitectura_pascal_contrato.py</code>	10	La escena, campo por campo, contra el esquema real del editor.
<code>test_editor_3d_puente_ui.py</code>	5	El intercambio de mensajes en el navegador, incluido el rechazo.
<code>test_plano_3d_e2e.py</code>	2	Que se vea: geometría dibujada con el editor real al otro lado del iframe.
<code>test_paperclip_estimaciones.py</code>	10	Las cuatro tablas se llenan en pantalla, con totales y aviso honesto.
<code>test_prewrite_order_adjunta_cotizacion.py</code>	8	La cotización llega con la tarea a quien la ejecuta.
<code>test_estimacion_agencia.py</code>	39	Estimación desde estructura, precedencia del integrador, proveedor y alias de clave.
<code>test_estimacion_prewrite_e2e.py</code>	10	De la agencia real al formulario real y de ahí a cada tabla de la base.

Además, `tests/features/pre_work_order.feature` recoge 7 escenarios Gherkin de la regla de negocio del reparto, conforme a la restricción R2 de `RESTRICCIONES_EXTREMAS.md`.

7. Puertas de calidad: evolución de las cifras

Fecha	Commit	pytest	GAS	Cobertura global
11 ago 2026	<code>abe11fc</code>	1 301 en verde	87 / 87	65.22 %
12 ago 2026	<code>2c141d0</code>	1 456 en verde	87 / 87	66.59 %
8 sep 2026	<code>fa409d8</code>	2 493 pasan · 6 saltadas	97 / 97	78.01 %
8 sep 2026	<code>4e8b9dd</code>	2 497 pasan · 6 saltadas	97 / 97	78.01 %
8 sep 2026	<code>5d18efc</code>	2 516 pasan · 10 saltadas	97 / 97	78.01 %
9 sep 2026	<code>5fc8cef</code>	2 558 pasan · 10 saltadas	97 / 97	78.39 %
9 sep 2026	<code>50c7330</code>	2 570 pasan · 10 saltadas	97 / 97	78.92 %

El piso de cobertura vigente es **60 %** y no se movió en todo el periodo: la mejora de 13.70 puntos es real, no un umbral rebajado. `ruff` en modo crítico (`E9,F63,F7,F82`) terminó limpio en cada hito, `xenon` en verde, y el inventario de `ruff` completo se mantuvo en **51 hallazgos preexistentes** —ni uno más

—. Las 10 pruebas saltadas corresponden en su mayoría al recorrido 3D de extremo a extremo, que requiere el editor servido en `HOLTMONT_EDITOR_URL`.

Nota de honestidad (obligación 4 de `CLAUDE.md`). Este informe es documentación: no modifica código de `api/`, `backend/` ni `tests/`. La suite **no se ejecutó en la sesión que lo redactó**, porque el contenedor no tiene instaladas las dependencias del proyecto (`pytest` y `langchain_core` ausentes). Todas las cifras de la tabla anterior son la salida literal registrada en el mensaje de cada commit por la sesión que sí las corrió; ninguna se estimó ni se redondeó aquí.

8. Deuda conocida y pendientes

Pendiente	Detalle y estado
DDL de los bloques F, G y H	Viáticos, transporte e ingeniería ya están mapeados en <code>TABLAS_W0</code> y su esquema está escrito en <code>docs/DDL_W0_BLOQUES.sql</code> (del PR #129, anterior a este periodo). Conviene verificar que el DDL esté aplicado en Supabase: si la tabla no existe, PostgREST responde <code>PGRST205</code> y el bloque se queda sólo en la nota de Obsidian, con su aviso.
Recorrido 3D en CI	<code>test_plano_3d_e2e.py</code> se salta sin el editor servido. Es la decisión correcta — no dar por bueno lo que no se comprobó— pero significa que la comprobación “¿se ve?” no corre automáticamente : hay que levantar el editor y correrla a mano.
Dos repositorios acoplados	La mitad de los arreglos del Hito 4 vive en <code>holtmont-3d-editor</code> . El contrato entre ambos está fijado por <code>test_arquitectura_pascal_contrato.py</code> de este lado y por el Zod real del otro, con <code>scripts/generar_escenas_pascal.py</code> como puente — pero son dos despliegues que hay que versionar juntos .
51 hallazgos de <code>ruff</code> completo	Deuda preexistente, estable en todo el periodo. Ningún cambio del módulo la aumentó ni bajó ninguna puerta para pasar (Directiva Cero).
Dependencia de proveedor externo	La agencia necesita clave de Google o de Groq. Sin ninguna, el mensaje ahora nombra las dos, y <code>/api/paperclip/diagnostico</code> permite distinguir “falta clave” de “falta librería” de “modelo retirado”.

Anexo A — Contrato de mensajes con el editor 3D

Mensaje	Dirección	Contenido y efecto
PASCAL_READY	editor → plataforma	El oyente del editor ya está montado. Dispara el envío del diseño ; sustituyó al retraso fijo que perdía mensajes.
HOLTMONT_3D_IMPORT	plataforma → editor	El diseño viaja en <code>projectData</code> (no <code>payload</code>), desenvuelto con <code>JSON.parse/JSON.stringify</code> para no chocar con el Proxy reactivo de Vue.
HOLTMONT_3D_IMPORT_ACK	editor → plataforma	Número de nodos aplicados y lista de descartes. La plataforma avisa si el diseño se dibujó incompleto.
HOLTMONT_3D_IMPORT_ERROR	editor → plataforma	La escena no dejó ningún nodo en pie: se conserva la escena anterior en vez de sustituirla por una vacía, y la plataforma marca el diseño en error.
HOLTMONT_3D_EXPORT	editor → plataforma	El diseño terminado, leído por la plataforma en <code>data.data</code> —que es lo que el fork manda—.

Del lado del editor, el puente `apps/editor/app/holtmont-bridge.tsx` valida cada nodo contra el esquema `AnyNode` (`apps/editor/lib/holtmont-import.ts`), rellena los valores por defecto que `setScene` no rellena, y resuelve los muebles: el generador manda el nombre en `metadata.holtmontAsset` y el puente lo busca en `CATALOG_ITEMS`, porque el catálogo de modelos 3D vive allá y no aquí.

Anexo B — Claves, proveedores y diagnóstico

Nombres de variable reconocidos

```
ALIAS_CLAVE_GEMINI = ("GEMINI_API_KEY", "Gemini_Key", "GEMINI_KEY",  
                     "GOOGLE_API_KEY", "GOOGLE_GEMINI_API_KEY")  
ALIAS_CLAVE_GROQ   = ("GROQ_API_KEY", "Groq_Key", "GROQ_KEY")
```

El orden es el de preferencia: primero el nombre canónico, luego los que se usan en la práctica. Si están los dos, gana el canónico.

Reparto de trabajo según las claves presentes

Claves disponibles	Comportamiento
Groq + Gemini	Reparto original con el que se afinaron los prompts: Groq arma el JSON de las tablas, Gemini escribe los reportes.
Sólo Gemini	Gemini corre todos los agentes; también sabe hacer <code>with_structured_output</code> .
Sólo Groq	Groq corre todos los agentes.
Librería ausente	La clave cuenta como ausente : antes se llegaba a construir el LLM y reventaba.
Ninguna	El mensaje de error nombra las dos claves, no una.

Qué responde el diagnóstico

`GET /api/paperclip/diagnostico` devuelve, por cada proveedor: si la clave está configurada, **con qué nombre de variable se encontró**, su prefijo (nunca la clave), si la librería entró en el bundle y qué

modelo usaría; más quién haría cada mitad del trabajo y si los clientes de texto y estructurados se pudieron construir de verdad. Se abre en el navegador y no consume una sola llamada al modelo.

Anexo C — Índice de commits del periodo

Commit	Fecha	PR	Asunto
861e5e4	11 ago 2026	#132	Devuelve la Pre Work Order, su IA y su editor 3D al frontend
abe11fc	11 ago 2026	#134	La Pre Work Order reparte su programa en el Tracker
2c141d0	12 ago 2026	#134	El Plano 2D se calcula, ya no se ilustra
fa409d8	8 sep 2026	#174	El Plano 3D dibuja lo que se le pidió, y avisa cuando no puede
4e8b9dd	8 sep 2026	#174	Una prueba que mira el plano 3D, no el store
94aab01	8 sep 2026	#174	Devuelve las capturas de verificación a su versión anterior (<i>mantenimiento</i>)
5d18efc	8 sep 2026	#175	La estimación de Paperclip vuelve a las tablas, y la cotización viaja con la orden
6481197	9 sep 2026	#176	Las tablas de la Pre Work Order se llenan aunque el Integrador falle
5fc8cef	9 sep 2026	#176	La Agencia Paperclip corre con la clave de Google, que es la que hay
50c7330	9 sep 2026	#176	La clave de Google se llama <code>Gemini_Key</code> , y el código sólo miraba <code>GEMINI_API_KEY</code>
3b4208a	9 sep 2026	#176	Las capturas de verificación vuelven a la versión de main (<i>mantenimiento</i>)

Informe generado el 21 de septiembre de 2026 a partir del historial de `git` del repositorio `Luis-Dokkaebi/HOLTMONT-PYTHON`. Fuente reproducible: `docs/informe_pre_work_order.html`.